INDIAN STOCK MARKET DIRECTION PREDICTION USING MACHINE LEARNING AND DEEP LEARNING

A PROJECT

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

BY

Arindam Pal (BCA/40002/24)

Sujal Kumar (BCA/40005/24)

Khushi Kumari (BCA/40016/24)

Sneha Tiwari (BCA/40020/24)

UNDER THE GUIDANCE OF

Dr. Binod Kumar Sharma

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BIRLA INSTITUTE OF TECHNOLOGY, MESRA

(LALPUR UNIT), RANCHI, JHARKHAND, 834001

(2026)

Certificate

This is to certify that the contents of this project entitled "Indian Stock Market Direction Prediction Using Machine Learning and Deep Learning" is a Bonafide work carried out by Arindam Pal (BCA/40002/24), Sujal Kumar (BCA/40005/24), Khushi Kumari (BCA/40016/24), Sneha Tiwari (BCA/40020/24) in partial fulfillment of the requirement for the award of Bachelor of Computer Applications under my guidance.

This is to further certify that the matter of the project has not been submitted by anyone else for the award of any degree.

(Dr. Binod Kumar Sharma)
Department of Computer Science and Engineering
Birla Institute of Technology, Mesra (Lalpur Unit)
Ranchi, Jharkhand, 834001

Certificate of Approval

The foregoing project is hereby approved as a creditable work on "Indian Stock Market Direction Prediction Using Machine Learning and Deep Learning", carried out and presented in the manner satisfactory to warrant its acceptance as prerequisite to the degree for which it has been entitled.

It is understood that by this approval the undersigned do not endorse any statement made or opinion expressed but approve work for the purpose for which it is submitted.

Committee for evaluation of the project

External Examiner | Internal Examiner

Date: | Date:

Professor In charge
Birla Institute of Technology, Mesra
(Lalpur Unit) Ranchi, Jharkhand, 834001

ACKNOWLEDGEMENT

We wish to record our sincere gratitude to our guide, Dr. Binod Kumar Sharma, Department of Computer Science and Engineering, Birla Institute of Technology, Mesra (Lalpur Unit), for his guidance throughout the course of this project. His insistence on methodological rigour, in particular on validating a time series model in a manner that respects the order of time, shaped the direction of this work more than any other single factor, and the negative result reported in this report is a direct consequence of following that advice.

We thank the Department of Computer Science and Engineering for providing the computing facilities and the academic environment in which this project was carried out.

We also acknowledge the open source community whose libraries this project depends upon, in particular the maintainers of scikit-learn, XGBoost, TensorFlow, pandas, pandas-ta, Plotly, Streamlit and yfinance. The project would not have been feasible within a single semester without them.

Finally, we thank our families and classmates for their encouragement and patience during the preparation of this work.

Roll	Name	Signature / Date
1	Arindam Pal (BCA/40002/24)	
2	Sujal Kumar (BCA/40005/24)	
3	Khushi Kumari (BCA/40016/24)	
4	Sneha Tiwari (BCA/40020/24)	

ABSTRACT

This project investigates whether the direction of the next trading day's close can be predicted for the Indian equity market using machine learning and deep learning models trained on historical price data. The system, named Disha, formulates the task as a three class classification problem, predicting whether an instrument will close UP, DOWN or NEUTRAL relative to a volatility aware threshold, and converts that prediction into a BUY, SELL or HOLD recommendation subject to a minimum confidence requirement.

Validation used an expanding window walk forward scheme of four folds, each validating on a distinct market regime between 2020 and 2023, followed by a single evaluation on a sealed test window covering January 2024 onward. Accuracy was compared against a majority class baseline rather than random chance, and strategy performance was assessed with a transaction cost of 0.05 percent charged on every change of position. Across twenty four model and instrument combinations, sixteen fell within two standard errors of the majority class baseline and were therefore statistically indistinguishable from it, eight exceeded it by more than two standard errors, and none fell significantly below it. Macro averaged F1 score was approximately 1.91 times that of the constant baseline, indicating that the models do recover genuine class structure that raw accuracy conceals. However, only seven of twenty four combinations exceeded a simple buy and hold strategy on risk adjusted return after transaction costs. Increasing model complexity did not produce a consistent improvement, and the LSTM recorded the lowest mean accuracy of the three model families while consuming approximately ninety three percent of total training time.

Keywords: Stock market prediction, direction classification, Random Forest, XGBoost, Long Short Term Memory, walk forward validation, technical indicators, backtesting, NIFTY 50, National Stock Exchange.

TABLE OF CONTENTS

Certificate ..... **ii**
Certificate of Approval ..... **iii**
Acknowledgement ..... **iv**
Abstract ..... **v**
Table of Contents ..... **vi**
List of Figures and Tables ..... **viii**

CHAPTER 1 INTRODUCTION **1**
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 2
1.4 Research Question 2
1.5 Objectives 3
1.6 Scope and Limitations 3
CHAPTER 2 LITERATURE REVIEW **5**
2.1 Deep Learning Approaches to Stock Prediction 5
2.2 Gradient Boosting and Hybrid Forecasting Models 5
2.3 Comparative Studies on the Indian Equity Market 5
2.4 Foundations of the Random Forest Algorithm 6
2.5 Empirical Behaviour of XGBoost 6
2.6 Technical Indicators and the RSI 6
2.7 Summary of Literature and Research Gap 7
CHAPTER 3 SYSTEM ANALYSIS **9**
3.1 Analysis of Prevailing Approaches 9
3.2 Proposed System 9
3.3 Feasibility Study 10
3.4 Requirement Specification 10
3.4.1 Functional Requirements 11
3.4.2 Non Functional Requirements 11
3.5 Hardware and Software Requirements 12
CHAPTER 4 SYSTEM DESIGN **13**
4.1 Architectural Overview 13
4.2 Data Flow Through the System 14
4.3 Module Decomposition 14
4.4 The Instrument Abstraction 15
4.5 The Model Abstraction Layer 16
4.6 Repository Structure 16

CHAPTER 5 METHODOLOGY **18**
5.1 Data Acquisition 18
5.2 Data Cleaning 18
5.3 Feature Engineering 18
5.3.1 Technical Indicators 18
5.3.2 Scale Invariance 19
5.4 Macroeconomic Features and Leakage Control 20
5.5 Target Definition 20
5.6 Walk Forward Validation Design 21
5.7 Random Forest Configuration 21
5.8 XGBoost Configuration 22
5.9 LSTM Configuration 22
5.10 Backtesting Engine 23
5.11 Recommendation Layer and Confidence Gate 24
CHAPTER 6 IMPLEMENTATION **25**
6.1 Pipeline Orchestration 25
6.2 Dashboard Architecture 25
6.3 Instrument Universe Management 26
6.4 On Demand Model Training 27
6.5 The Loading Interface 28
6.6 Multi Resolution Charting 29
6.7 Technical Screener 29
6.8 Caching Strategy 31
CHAPTER 7 RESULTS AND DISCUSSION **32**
7.1 Exploratory Data Analysis 32
7.2 Walk Forward Fold Results 33
7.3 Sealed Test Set Results 33
7.4 Statistical Significance Against the Baseline 35
7.5 Confusion Matrix Analysis 37
7.6 Trading Performance After Costs 37
7.7 Effect of Model Complexity 38
7.8 Computational Cost 39
7.9 Consolidated Observations 39
CHAPTER 8 CONCLUSION **41**
8.1 Summary of Work 41
8.2 Answer to the Research Question 41
8.3 Limitations 42
8.4 Future Scope 42
BIBLIOGRAPHY **44**

LIST OF FIGURES

Figure 4.1 End to end system architecture 13
Figure 4.2 Pipeline and dual model design as presented inside the application 14
Figure 5.1 Expanding window walk forward validation scheme 21
Figure 6.1 Main dashboard showing the prediction for the next session 26
Figure 6.2 Untrained instrument state offering quick and full training 28
Figure 6.3 Disha loading interface during a training run 29
Figure 6.4 Price chart with moving averages and relative strength index 29
Figure 6.5 Technical screener showing the RSI screen and breakout table 30
Figure 6.6 Bollinger band breakout chart with breaching closes circled 30
Figure 7.1 Track record of settled calls against realised outcomes 34
Figure 7.2 Model comparison on the sealed test set 35

LIST OF TABLES

Table 2.1 Summary of reviewed literature and relation to this project 7
Table 3.1 Functional requirements 11
Table 3.2 Software environment and library versions 12
Table 4.1 Module decomposition and responsibilities 14
Table 5.1 Scale invariant features derived for every instrument 19
Table 5.2 Walk forward fold definitions and market regimes 21
Table 5.3 Random Forest hyperparameters 21
Table 5.4 XGBoost hyperparameters 22
Table 5.5 LSTM architecture and training configuration 22
Table 6.1 Dashboard views and their data dependencies 26
Table 7.1 Class balance of the NIFTY 50 labelled dataset 32
Table 7.2 Walk forward fold results for the NIFTY 50 index 33
Table 7.3 Sealed test set results, NIFTY 50 index (689 observations) 33
Table 7.4 Sealed test set accuracy across all eight instruments 35
Table 7.5 Statistical comparison against the majority class baseline 36
Table 7.6 Confusion matrix, Random Forest on the NIFTY 50 sealed test set 37
Table 7.7 Risk adjusted performance against buy and hold (annualised Sharpe) 37
Table 7.8 Mean performance by model family across eight instruments 38
Table 7.9 Measured training time per model family 39

CHAPTER 1

INTRODUCTION

1.1 Background

The National Stock Exchange of India is among the largest exchanges in the world by number of trades, and the NIFTY 50 index, which tracks fifty of the largest and most liquid companies listed on it, is the most widely followed barometer of Indian equity market sentiment. The question of whether future price movement on such an exchange can be anticipated from information already contained in past prices is one of the oldest questions in quantitative finance, and it remains contested.

The efficient market hypothesis, in its weak form, asserts that all information contained in historical prices is already reflected in the current price, and that no trading rule based solely on past prices can therefore produce consistent excess returns. Against this stands a very large body of applied work, much of it published in the last decade, reporting that machine learning and deep learning models can predict stock movement with accuracies well above chance. The apparent contradiction between these two positions is not merely theoretical. It has direct consequences for how such a model should be built, validated and reported.

This project was conceived as a direct examination of that gap. Rather than seeking the highest achievable accuracy figure, the work sets out to determine what accuracy survives when the validation design is made deliberately strict, and to report that figure whether or not it is favourable.

1.2 Motivation

Three considerations motivated the choice of this problem.

First, the problem is genuinely difficult and genuinely unresolved, which makes it a more instructive subject for a final semester project than a problem with a known correct answer. The outcome of the investigation was not predetermined at the point the project began.

Third, the project permits a controlled study of model complexity. Random Forest, XGBoost and Long Short Term Memory networks represent three distinct and

increasingly sophisticated approaches to supervised learning: bootstrap aggregation of decision trees, sequential gradient boosting of weak learners, and gated recurrent neural networks able to model temporal dependence directly. Running all three through an identical data pipeline, with identical features, identical splits and identical evaluation, isolates the effect of model architecture from every other variable.

1.3 Problem Statement

The system is required to answer a single question for a given listed instrument: will the instrument close higher, lower, or approximately unchanged on the next trading day relative to its present close?

The third class, NEUTRAL, is necessary rather than cosmetic. Equity prices almost never close exactly unchanged, so a strictly binary formulation would classify a movement of two basis points as a genuine upward move and would require the model to resolve what is effectively measurement noise. A threshold band around zero absorbs these movements. The band is set at plus or minus 0.25 percent for the index and plus or minus 0.50 percent for individual stocks, reflecting the observation that a single company is approximately twice as volatile on a daily basis as the index of which it is a constituent.

The prediction must then be translated into an actionable recommendation of BUY, SELL or HOLD, subject to the requirement that the system declines to commit when its own confidence is insufficient to justify a position.

1.4 Research Question

The project is organised around one primary research question and one secondary question that follows from the multi instrument design.

Primary: Does increasing model complexity, proceeding from Random Forest to XGBoost to Long Short Term Memory networks, improve predictive accuracy and trading performance on the Indian stock market, when all three are evaluated under an identical and temporally correct validation scheme?

Secondary: Is an individual stock easier or harder to predict than the index of which it is a constituent? The pipeline runs identically on both, so the comparison is made on equal terms.

1.5 Objectives

The following objectives were defined at the outset of the project.

1. To construct a reproducible data pipeline that acquires daily price history for the NIFTY 50 index and its constituent companies, cleans it, and derives a consistent set of technical features.
2. To define a labelling scheme that converts next day returns into three directional classes using a volatility aware threshold.
3. To implement an expanding window walk forward validation scheme that respects the chronological ordering of financial data and never permits a model to train on observations that postdate its test window.
4. To implement Random Forest, XGBoost and LSTM classifiers behind a common interface so that they can be substituted for one another without modification to the surrounding pipeline.
5. To evaluate all models against a majority class baseline rather than against random chance, and to report the statistical significance of any difference.
6. To implement a backtesting engine that converts predictions into positions and charges a realistic transaction cost on every change of position.
7. To develop an interactive dashboard that exposes the current prediction, the historical track record, the comparative model results and the methodology to a non specialist user.
8. To document the findings honestly, including the possibility that no model demonstrates a durable predictive advantage.

1.6 Scope and Limitations

The scope of the project is bounded as follows. The system operates on daily bars only. It predicts one trading day ahead and no further. It uses price derived technical features together with two macroeconomic series, and does not incorporate news text, analyst estimates, order book depth, corporate filings or any other alternative data. It covers the NIFTY 50 index and the fifty constituent companies of that index, with the facility for a user to add any symbol available from the data provider.

The following limitations are acknowledged explicitly. The backtest assumes that a position can be taken at the closing price of the day on which the signal is generated, which ignores slippage and the practical difficulty of executing at the close. Short selling is assumed to be available without borrowing constraints or cost beyond the stated transaction charge. The transaction cost of 0.05 percent is a simplification that does not separate brokerage, exchange charges, securities transaction tax and goods and services tax. Dividends are handled through the use of adjusted prices rather than modelled explicitly. Finally, several constituents of the index listed after 2015 and therefore possess shorter histories than the remainder, which the system reports but does not otherwise compensate for.

CHAPTER 2

LITERATURE REVIEW

This chapter surveys the published work most directly relevant to the modelling choices made in this project. The selected references fall into three groups: studies applying deep learning and ensemble methods to stock forecasting, foundational descriptions of the two tree based algorithms employed, and work on the technical indicators used as model inputs. The chapter concludes by identifying the methodological gap this project addresses.

2.1 Deep Learning Approaches to Stock Prediction

Sunny, Maswood and Alharbi, in their work on deep learning based stock price prediction using LSTM and bi-directional LSTM models, examine the application of recurrent neural architectures to the stock forecasting problem. Long Short Term Memory networks are of particular relevance to the present project because they are explicitly designed to model sequential dependence, retaining information across many time steps through a gated cell state and thereby addressing the vanishing gradient problem that limits simple recurrent networks on long sequences. The bi-directional variant extends this by processing a sequence in both the forward and reverse directions.

2.2 Gradient Boosting and Hybrid Forecasting Models

Oukhouya, Kadiri, El Himdi and Guerbaz, in their study forecasting international stock market trends, consider XGBoost, LSTM and a combined LSTM-XGBoost formulation, and additionally address the backtesting of XGBoost models. This work is methodologically the closest of the reviewed references to the present project, in that it compares the same two principal model families and explicitly treats backtesting as part of the evaluation rather than reporting classification metrics alone.

2.3 Comparative Studies on the Indian Equity Market

Khandelwal, Borkar and Ukhalkar present a comparative analysis of LSTM, Random Forest and XGBoost models for stock price prediction in the Indian equity market. This reference is the most directly comparable to the present work, since it examines the same three model families on the same national market.

The present project differs from this reference in formulation and in evaluation standard. The task here is three class direction classification rather than price prediction, the comparison is made against a majority class baseline rather than against an absolute accuracy figure, and the test window is sealed and evaluated once rather than being used repeatedly during model selection.

2.4 Foundations of the Random Forest Algorithm

Salman, Kalakech and Steiti provide an overview of the Random Forest algorithm. Random Forest constructs an ensemble of decision trees, each fitted to a bootstrap resample of the training data and each considering a random subset of the available features at every split. Predictions are aggregated across the ensemble, which reduces the variance that a single deep decision tree would exhibit while retaining the ability to represent non linear interactions between features.

Two properties of the algorithm are directly relevant to this project. First, Random Forest is comparatively robust to irrelevant features, because a feature carrying no information is unlikely to be selected for a split when better alternatives are available in the random subset. Given that the present project supplies twenty six to twenty eight technical features, many of which are correlated with one another, this robustness is valuable. Second, the algorithm supports class weighting, which permits the majority NEUTRAL class to be prevented from dominating the fitted model. The configuration adopted in this project uses balanced subsample weighting for this reason.

2.5 Empirical Behaviour of XGBoost

Ramraj, Uzir, Sunil and Banerjee report on experiments applying the XGBoost algorithm to prediction and classification across different datasets. XGBoost implements gradient boosting, in which trees are fitted sequentially and each successive tree is trained to correct the residual errors of the ensemble constructed so far. Unlike Random Forest, where trees are independent and averaging reduces variance, boosting reduces bias by construction and is consequently more prone to overfitting if left unregularised.

2.6 Technical Indicators and the RSI

Sahin and Ozbayoglu propose a trend-normalized RSI indicator for stock trading systems using evolutionary computation. The Relative Strength Index is a bounded momentum oscillator that compares the magnitude of recent gains to the magnitude of recent losses over a lookback window, conventionally fourteen periods, producing a value between zero and one hundred. Readings above seventy are conventionally interpreted as indicating an overbought condition and readings below thirty an oversold condition.

2.7 Summary of Literature and Research Gap

Table 2.1: Summary of reviewed literature and relation to this project

Reference	Focus	Relation to this project
Sunny, Maswood and Alharbi	LSTM and bi-directional LSTM for stock price prediction	Justifies inclusion of a recurrent architecture as the high complexity model
Oukhouya, Kadiri, El Himdi and Guerbaz	XGBoost, LSTM, hybrid models and backtesting	Closest methodological parallel; supports pairing classification metrics with backtesting
Khandelwal, Borkar and Ukhalkar	Comparative LSTM, Random Forest and XGBoost on Indian equities	Most directly comparable study; same three families, same market
Salman, Kalakech and Steiti	Random Forest algorithm overview	Foundation for the baseline ensemble model and its class weighting
Ramraj, Uzir, Sunil and Banerjee	Empirical behaviour of XGBoost across datasets	Informs the regularisation strategy adopted after observed overfitting
Sahin and Ozbayoglu	Trend-normalized RSI for trading systems	Informs use and interpretation of RSI as a feature and in the screener

The gap this project addresses is not a gap in model architecture. It is a gap in evaluation standard. The present work contributes a study in which four methodological constraints are applied simultaneously and consistently across all three model families.

1. Validation proceeds by expanding window walk forward splits in strict chronological order, with each validation year drawn from a distinct market regime, so that no model is ever fitted on observations postdating its evaluation window.
2. A test window covering January 2024 onward is sealed and evaluated exactly once per model, so that no iterative tuning against the test set can occur.
3. Accuracy is compared against the majority class baseline for each instrument rather than against an assumption of balanced classes, which raises the threshold a model must exceed to claim an advantage.

4. Trading performance is assessed with a transaction cost charged on every change of position, so that high turnover strategies are penalised as they would be in practice.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Analysis of Prevailing Approaches

Before specifying the proposed system it is useful to state what it is intended to improve upon. Three characteristics recur in publicly available implementations of stock prediction systems and each introduces a specific defect.

Random train and test splitting. Applying a randomised split to a time ordered series places future observations in the training set and past observations in the test set. The resulting model has effectively been shown the answers to the questions on which it is subsequently examined. Reported accuracies obtained this way are not estimates of future performance.

Comparison against an assumed uniform baseline. A three class problem is frequently assessed against a figure of 33.3 percent on the assumption that the classes are equiprobable. Financial return classes are not equiprobable. In this project the NEUTRAL class accounts for approximately 27 percent of NIFTY 50 observations and the UP class for approximately 40 percent, so a trivial rule predicting the most frequent class every day already achieves close to 40 percent. A model reporting 35 percent accuracy is therefore worse than a rule requiring no model at all, despite appearing to exceed chance.

Omission of transaction costs. A directional model generates a position change whenever its prediction changes. At daily frequency this can produce several hundred position changes per year. Evaluating the resulting strategy without charging a cost for each change overstates its performance, and the overstatement grows with turnover, which is precisely the regime in which such models operate.

3.2 Proposed System

The proposed system is a complete pipeline from data acquisition through to an interactive user interface, organised so that each stage is independently inspectable and the artefacts of each stage persist on disk.

Two design decisions distinguish the proposed system from a conventional implementation. The first is the dual model artefact: for every combination of instrument and algorithm the system retains two separately fitted models, one trained only on data preceding the sealed test window and scored exactly once, and one refitted on all available history and never scored. All reported performance figures derive from the first; all current recommendations derive from the second. The second decision is the separation of data preparation from model training, which permits the price data for the entire instrument universe to be prepared in advance while leaving the computationally expensive training to be performed on demand for the instrument a user actually selects.

3.3 Feasibility Study

The project is technically feasible on commodity hardware. All data is obtained from a freely available public interface, the two tree based models fit within seconds on approximately 2,900 daily observations, and the recurrent model requires approximately thirty six seconds per fit using the central processing unit alone. No graphics processing unit is required and total disk consumption for the full instrument universe remains within a few hundred megabytes.

The project is economically feasible in that it incurs no licensing or subscription cost. Every component of the software stack is open source and the market data is obtained without charge, so the only resources consumed are development time and ordinary computing hardware.

The project is operationally feasible in that the application is delivered as a local web application requiring a single command to start. A user without programming knowledge can select an instrument, initiate training and read the resulting recommendation without interacting with the source code, and explanatory material describing the pipeline and the reasoning behind the two model variants is embedded in the interface itself.

3.4 Requirement Specification

The requirements are separated into functional requirements, which state what the system must do, and non functional requirements, which constrain how it must do it. The non functional requirements concerning temporal integrity are of particular

importance, since a violation of them would invalidate every performance figure the system produces while remaining invisible in its output.

3.4.1 Functional Requirements

Table 3.1 states the functional requirements of the system.

Table 3.1: Functional requirements

No.	Requirement
FR1	Acquire daily open, high, low, close and volume data for a specified instrument from January 2015 to the present.
FR2	Acquire the India VIX and USD to INR series as macroeconomic context, and the NIFTY 50 index as market context for individual stocks.
FR3	Derive a fixed set of scale invariant technical features from the acquired price data.
FR4	Assign each observation a directional label using an instrument specific threshold.
FR5	Train Random Forest, XGBoost and LSTM classifiers under an expanding window walk forward scheme.
FR6	Evaluate each trained model once on a sealed test window and persist the resulting metrics.
FR7	Refit each model on all available history and persist it separately for prediction use.
FR8	Produce a BUY, SELL or HOLD recommendation with an associated confidence value for the next session.
FR9	Downgrade any recommendation to HOLD when the leading class probability falls below the configured minimum.
FR10	Backtest predictions into a position series and report risk adjusted performance after transaction costs.
FR11	Present the recommendation, track record, comparative results and methodology through a graphical interface.
FR12	Permit a user to add any instrument available from the data provider and train models for it on demand.
FR13	Provide a screening facility filtering instruments by relative strength index and by Bollinger band breach.

3.4.2 Non Functional Requirements

Reproducibility. All stochastic components shall be seeded so that a given dataset and configuration produce identical results across runs.

Temporal integrity. No stage of the pipeline shall permit information from after a prediction point to influence that prediction. This constraint governs the labelling, the construction of macroeconomic features, the fold boundaries, the scaler fitting and the backtest alignment.

Responsiveness. Interface interactions that do not require model training shall complete without perceptible delay. This is achieved by caching derived artefacts and by preparing price data for the full universe in advance.

Transparency. The interface shall state the basis of every figure it displays, including which model variant produced it and over what window, and shall not present a recommendation without its associated confidence.

Maintainability. Each model shall implement a common abstract interface so that an additional model family can be introduced without modification to the pipeline, the evaluation code or the interface.

3.5 Hardware and Software Requirements

The system was developed and tested on a machine running Windows 11 with a multi core central processing unit and sixteen gigabytes of memory. No specialised hardware is required. Table 3.1 lists the software environment.

Table 3.2: Software environment and library versions

Component	Version	Role in the system
Python	3.12.10	Implementation language for the entire pipeline
pandas	3.0.5	Tabular data handling and time series resampling
NumPy	Bundled	Numerical array operations
scikit-learn	1.9.0	Random Forest, metrics, preprocessing utilities
XGBoost	3.4.1	Gradient boosted tree classifier
TensorFlow	2.21.0	Keras backend for the LSTM network
pandas-ta	Bundled	Technical indicator computation
yfinance	1.6.0	Market data acquisition
Streamlit	1.61.1	Interactive dashboard framework
Plotly	6.9.0	Interactive charting
joblib	Bundled	Model serialisation

The complete implementation comprises 3,869 lines of Python distributed across twenty one modules, excluding notebooks and generated artefacts.

CHAPTER 4

SYSTEM DESIGN

4.1 Architectural Overview

The system is organised into two independent executables that communicate only through files written to disk. The first is a batch pipeline that acquires data, derives features, trains models and persists artefacts. The second is an interactive dashboard that reads those artefacts and presents them. The dashboard performs no training within its own process; when a user requests training it launches the pipeline as a separate operating system process and monitors its progress by reading the log file that process writes.

Figure 4.1 presents the overall architecture. Rectangular stages denote processing steps and the labels beneath each arrow denote the artefact written at that stage.

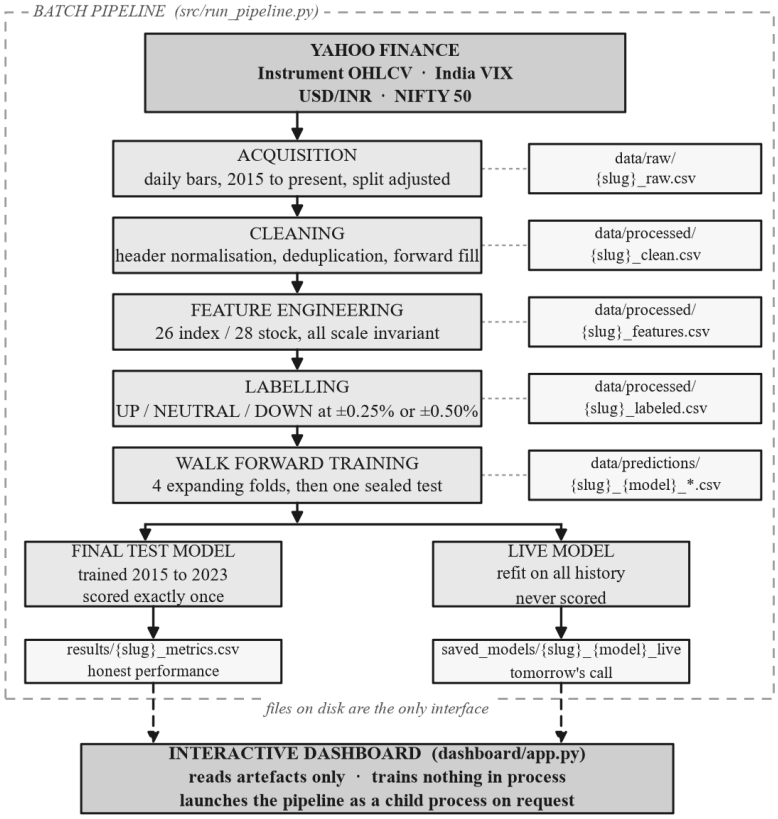


Figure 4.1: End to end system architecture

Figure 4.2 reproduces the summary of this design as it is presented to the user inside the application itself, which includes the division between the sealed evaluation model and the refitted prediction model.

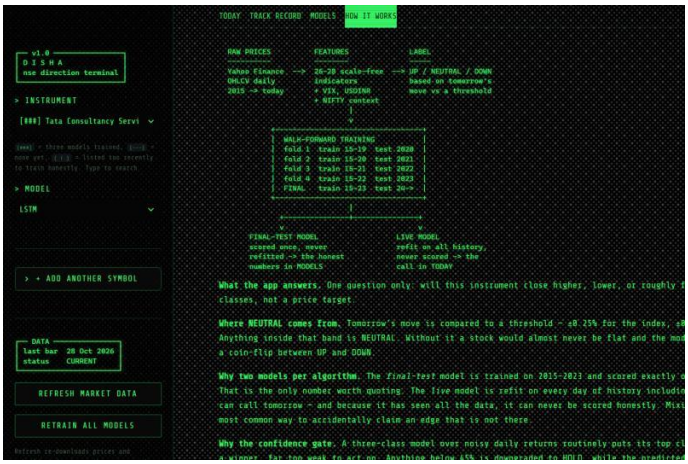


Figure 4.2: Pipeline and dual model design as presented inside the application

4.2 Data Flow Through the System

When the dashboard requires a current recommendation it performs no computation beyond loading the refitted model artefact, selecting the most recent row of the feature matrix, and executing a single forward pass. The absence of any acquisition, derivation or fitting at this point is what permits the interface to respond immediately.

4.3 Module Decomposition

Table 4.1 lists each module of the implementation together with its responsibility and its position in the pipeline.

Table 4.1: Module decomposition and responsibilities

Module	Lines	Responsibility
config.py	228	Paths, constants, instrument registry, feature list selection
data_loader.py	60	Acquisition of price and macroeconomic series
data_cleaner.py	64	Header normalisation, type coercion, deduplication, forward fill
features.py	169	Technical indicator computation and scale invariant transformation
target.py	77	Directional labelling and class balance reporting
validation.py	79	Walk forward fold definitions and the sealed test split
evaluate.py	192	Model construction, fold execution, metric computation, persistence
backtest.py	259	Position mapping, cost accounting, risk adjusted metrics
predict.py	107	Model loading and current prediction
recommend.py	115	Action mapping, confidence gate, indicator narrative

Module	Lines	Responsibility
run_pipeline.py	106	Command line orchestration of the full pipeline
prepare_universe.py	107	Bulk preparation of all instruments without training
models/base.py	76	Abstract model interface, probability alignment, sequence construction
models/rf_model.py	41	Random Forest implementation
models/xgb_model.py	55	XGBoost implementation
models/lstm_model.py	128	LSTM implementation with scaling and early stopping
dashboard/app.py	~800	Dashboard layout, views, caching, job launching
dashboard/retro.py	~620	Interface styling, ASCII rendering, loading interface
dashboard/screener.py	163	Cross sectional indicator screens

4.4 The Instrument Abstraction

Every instrument the system can process is represented by a single immutable record carrying its internal key, the symbol understood by the data provider, a display name, a kind of either index or stock, and the threshold defining its NEUTRAL band. Critically, this record also owns its own file paths, so that no other module constructs a path by string concatenation.

```
@dataclass(frozen=True)
class Instrument:
    key: str
    symbol: str
    name: str
    kind: str  # "index" or "stock"
    threshold: float

    @property
    def slug(self) -> str:
        return self.key.lower()

    def raw_path(self) -> Path:
        return RAW_DIR / f"{self.slug}_raw.csv"

    def clean_path(self) -> Path:
        return PROCESSED_DIR / f"{self.slug}_clean.csv"

    def features_path(self) -> Path:
        return PROCESSED_DIR / f"{self.slug}_features.csv"

    def labeled_path(self) -> Path:
        return PROCESSED_DIR / f"{self.slug}_labeled.csv"
```

Listing 4.1: The immutable instrument record, which owns its own artefact paths

The set of features supplied to a model is likewise determined by the instrument, through a single function that both the training code and the prediction code call. This

is what guarantees that the column order presented at prediction time is identical to the column order on which the model was fitted.

```
def model_features_for(instrument: Instrument) -> list[str]:
    if instrument.kind == "stock":
        return MODEL_FEATURES + MARKET_FEATURES   # 28 columns
    return list(MODEL_FEATURES)                   # 26 columns
```

Listing 4.2: Feature selection, the single point at which the feature count is decided

4.5 The Model Abstraction Layer

All three model families implement a common abstract interface. This permits the evaluation code to treat them uniformly and allows a further family to be added without modification elsewhere.

```
class DirectionModel:
    name: str
    lookback: int = 1          # 1 = tabular, > 1 = requires a sequence

    def fit(self, X, y): ...
    def predict_proba(self, X, context=None): ...
    def predict(self, X, context=None): ...
    def save(self, stem): ...
    def load(stem): ...
```

Listing 4.3: The abstract model interface shared by all three families

The lookback attribute is the mechanism by which sequential and tabular models coexist behind one interface. A model declaring a lookback of one receives a single row of features. A model declaring a larger lookback additionally receives a context argument containing the preceding rows, which it assembles internally into a sequence tensor. The surrounding pipeline does not branch on model type; it merely supplies the context when the model requests it.

```
def align_proba(model_classes, proba) -> np.ndarray:
    """Pad a probability matrix to all three classes in order 0, 1, 2.

    A fold with no DOWN days yields a two column matrix. Reading column 1
    positionally would report UP as NEUTRAL, which is silent and wrong.
    """
    full = np.zeros((proba.shape[0], N_CLASSES))
    for position, cls in enumerate(model_classes):
        full[:, int(cls)] = proba[:, position]
    return full
```

Listing 4.4: Probability alignment, which prevents silent class mislabelling

4.6 Repository Structure

```
dishanifty/
    src/
        config.py  data_loader.py  data_cleaner.py  features.py
        target.py  validation.py   evaluate.py      backtest.py
        predict.py recommend.py    track_record.py  run_pipeline.py
        prepare_universe.py        fetch_universe.py
        models/
            base.py  rf_model.py  xgb_model.py  lstm_model.py
    dashboard/
        app.py  retro.py  screener.py  timeframes.py
    data/
        raw/         processed/      predictions/
        nifty50_constituents.json    universe_status.json
    results/         saved_models/   logs/   notebooks/
```

Listing 4.5: Repository structure

CHAPTER 5

METHODOLOGY

This chapter documents each stage of the pipeline in the order in which it executes, stating the design decision taken at each point and the reasoning behind it.

5.1 Data Acquisition

Daily price history is retrieved through the yfinance interface to Yahoo Finance. For each instrument the open, high, low, close and volume series are requested from 1 January 2015 to the present date, with no fixed end date so that the test window extends automatically as time passes.

```
df = yf.download(instrument.symbol, start=start, end=end,
                 auto_adjust=True, progress=False)
```

Listing 5.1: Price acquisition

The auto_adjust parameter is not optional. With adjustment disabled, a corporate action such as a stock split appears in the series as an abrupt single day movement of large magnitude. During development this was observed directly: the Tata Consultancy Services series exhibited an apparent fall of approximately fifty percent on its split date. A model trained on such a series learns that one specific historical date was catastrophic, which conveys nothing about future behaviour and corrupts every volatility derived feature in the surrounding window. Adjusted prices restore continuity across splits and dividend distributions.

5.2 Data Cleaning

5.3 Feature Engineering

Feature engineering proceeds in two stages. The first computes conventional technical indicators from the cleaned price series. The second converts them into scale invariant form, which is the single most consequential decision in the data preparation pipeline.

5.3.1 Technical Indicators

The indicators computed are the daily return, rolling volatility over ten and twenty sessions, simple and exponential moving averages at ten, twenty and fifty sessions, the

fourteen period Relative Strength Index, the Moving Average Convergence Divergence with its signal and histogram at the conventional twelve, twenty six and nine settings, Bollinger Bands at twenty periods and two standard deviations, the fourteen period Average True Range, and ten period momentum. Indicator output columns are selected by prefix rather than exact name, because the underlying library varies its suffix conventions between releases.

5.3.2 Scale Invariance

Raw price levels are never presented to a model. Every feature is expressed as a ratio, a percentage or a bounded oscillator. The reasoning is as follows. A decision tree fitted on data from 2015 may learn a split condition such as a fifty session moving average below nine thousand. That condition carried information in 2015. Once the same series trades at twenty four thousand the condition is never satisfied again and the learned rule is inert. Because the validation scheme is an expanding window in which early folds train only on early data, any feature expressed in absolute price units would degrade systematically as the window advances.

Table 5.1: Scale invariant features derived for every instrument

Feature	Definition	Rationale
Return_Lag1 to Lag3	Returns of the previous three sessions	Short horizon autocorrelation
Return_5d	Five session cumulative return	Week scale momentum
Close_vs_SMA10 / 20 / 50	Close divided by simple moving average	Position relative to trend
Close_vs_EMA10 / 20 / 50	Close divided by exponential moving average	Position relative to weighted trend
SMA10_vs_SMA20	Ratio of two moving averages	Short term trend alignment
SMA20_vs_SMA50	Ratio of two moving averages	Medium term trend alignment
MACD_Norm	MACD divided by close	Convergence signal, scale free
MACD_Hist_Norm	MACD histogram divided by close	Rate of change of convergence
BB_Position	Position within the Bollinger band, zero to one	Relative extension
BB_Width	Band width divided by close	Volatility regime
ATR_Pct	Average true range divided by close	Realised range as a proportion
Momentum_10_Pct	Ten session momentum divided by close	Momentum, scale free
Volume_Ratio	Volume divided by its twenty session average	Participation relative to normal
RSI_14	Fourteen period relative strength index	Already bounded zero to one hundred

Feature	Definition	Rationale
Volatility_10 / 20	Rolling standard deviation of returns	Already a ratio
Market_Return	Same day NIFTY 50 return (stocks only)	Contemporaneous market context
Market_Return_5d	Five session NIFTY 50 return (stocks only)	Market trend context

5.4 Macroeconomic Features and Leakage Control

The direction of the fill is a correctness requirement rather than a convenience. A forward fill propagates a past observation onto a later date. It cannot move a future observation backward. Consequently no information that postdates a given prediction point can enter that point's feature vector. A backward fill or an interpolation at this stage would silently invalidate every performance figure the system subsequently produces, and would do so in a manner that is invisible in the output. This constraint is recorded in the module documentation so that any future extension of the macroeconomic feature set preserves it.

5.5 Target Definition

The label attached to each observation describes the movement from that observation's close to the following session's close.

```
df["Next_Return"] = df["Close"].pct_change().shift(-1)

df["Target"] = 1                                          # NEUTRAL
df.loc[df["Next_Return"] >  threshold, "Target"] = 2    # UP
df.loc[df["Next_Return"] < -threshold, "Target"] = 0    # DOWN

df = df.dropna(subset=["Next_Return"])
```

Listing 5.2: Directional labelling

The negative shift is what makes this a forecasting problem rather than a description of the present. A row dated t carries features that were observable at the close of t and a label describing the move from t to t plus one. The final row of every series is discarded because its successor has not yet occurred and it therefore cannot be labelled. This also explains why the labelled file contains exactly one fewer observation than the feature file, and why the most recent feature row is available as a prediction input while never appearing as a training example.

5.6 Walk Forward Validation Design

Table 5.2: Walk forward fold definitions and market regimes

Fold	Training window	Validation year	Market regime
Fold 1	2015 to 2019	2020	COVID-19 crash and recovery
Fold 2	2015 to 2020	2021	Post-pandemic recovery rally
Fold 3	2015 to 2021	2022	Global interest rate tightening
Fold 4	2015 to 2022	2023	Stabilisation
Final test	2015 to 2023	2024 onward	Sealed, evaluated once per model

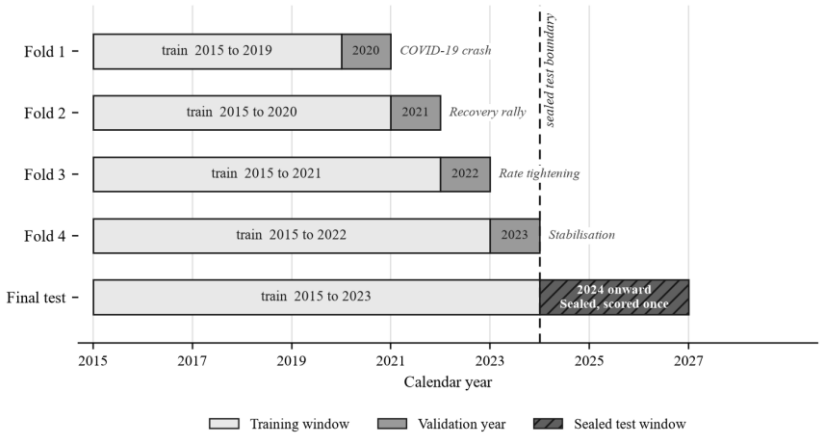


Figure 5.1: Expanding window walk forward validation scheme

All observations from 1 January 2024 onward constitute the sealed test window. A single function opens this window and the training routine invokes it exactly once per model family. The purpose of the seal is to prevent the iterative tuning that would otherwise occur: a test set consulted repeatedly during configuration selection ceases to be an unbiased estimate of generalisation, because the configuration eventually selected is the one that happens to suit that particular stretch of history.

5.7 Random Forest Configuration

The Random Forest classifier was configured with deliberate constraints on tree depth and leaf size, in recognition that the signal to noise ratio in daily financial returns is low and an unconstrained ensemble will fit noise.

Table 5.3: Random Forest hyperparameters

Parameter	Value	Justification
n_estimators	500	Variance reduction through averaging
max_depth	8	Constrains interaction order; deliberate underfitting

Parameter	Value	Justification
min_samples_leaf	25	Prevents leaves fitted to a handful of noisy observations
max_features	sqrt	Decorrelates trees across correlated technical features
class_weight	balanced_subsample	Prevents the majority class from dominating
random_state	42	Reproducibility

```
RandomForestClassifier(
    n_estimators=500,
    max_depth=8,
    min_samples_leaf=25,
    max_features="sqrt",
    class_weight="balanced_subsample",
    random_state=42,
    n_jobs=-1,
)
```

Listing 5.3: Random Forest configuration

5.8 XGBoost Configuration

The XGBoost configuration is conspicuously conservative, and arrived at that state empirically. An initial configuration using four hundred estimators at a maximum depth of four produced a training accuracy of 0.88 against a validation accuracy of approximately 0.35. That gap is not a tuning inefficiency; it is memorisation of the training window. The configuration was progressively constrained until the gap closed.

Table 5.4: XGBoost hyperparameters

Parameter	Value	Justification
n_estimators	150	Reduced from 400 after observed overfitting
learning_rate	0.02	Small steps limit the contribution of any single tree
max_depth	2	Reduced from 4; permits only shallow interactions
subsample	0.6	Row sampling introduces stochastic regularisation
colsample_bytree	0.5	Column sampling decorrelates successive trees
min_child_weight	40	Requires substantial support for any split
reg_lambda	20.0	Strong L2 penalty on leaf weights
reg_alpha	1.0	L1 penalty encouraging sparsity
gamma	0.5	Minimum loss reduction required to split

5.9 LSTM Configuration

Table 5.5: LSTM architecture and training configuration

Component	Setting	Justification
Lookback	30 sessions	Approximately six trading weeks of context
Layer 1	LSTM, 64 units, sequences returned	Primary temporal encoder
Dropout 1	0.2	Regularisation between recurrent layers
Layer 2	LSTM, 32 units	Compression to a fixed length representation
Dropout 2	0.2	Regularisation before the dense head
Dense	16 units, ReLU activation	Non linear combination
Output	3 units, softmax activation	Class probability distribution
Optimiser	Adam, learning rate 0.001	Standard adaptive optimiser
Loss	Sparse categorical cross entropy	Integer class labels
Epochs	60, early stopping patience 8	Best weights restored
Batch size	32	Standard for sequence data of this length
Gradient clipping	5.0	Guards against exploding gradients
Scaling	StandardScaler	See discussion below
Seed	42	Reproducibility

Two decisions in this configuration merit explanation.

Choice of scaler. Standardisation was used rather than min-max normalisation. Daily return distributions are heavy tailed. A min-max transformation derives its bounds from the extreme observations, so a single crash day compresses the ordinary ninety nine percent of observations into a narrow portion of the output range, destroying the resolution the network requires. Standardisation is governed by the mean and standard deviation and is far less sensitive to isolated extremes. The scaler is fitted inside the training routine on training data only, never on the full series.

Early stopping split. The validation subset used by early stopping is taken as the final fifteen percent of the training window in chronological order, never by random sampling. Shuffling at this point would reintroduce precisely the temporal leakage that the outer fold structure exists to prevent, and would do so invisibly within a single fold.

5.10 Backtesting Engine

The backtesting engine converts a prediction series into a position series, applies that series to realised returns, charges a transaction cost on every change of position, and computes risk adjusted performance metrics.

```
TRANSACTION_COST = 0.0005        # 0.05 percent per position change
TRADING_DAYS_PER_YEAR = 252
```

```
POSITION_MAP = {0: -1, 1: 0, 2: 1}   # DOWN short, NEUTRAL cash, UP long
```

Listing 5.4: Backtest constants

The alignment of positions to returns is the point at which lookahead would most easily be introduced, and is handled explicitly.

```
aligned["Position"] = aligned["Predicted_Target"].map(POSITION_MAP)

# A signal decided using information available at t is applied to the
# return realised from t to t+1, so the position is shifted forward one
# day. Without this shift the backtest would act on its own answer.
applied_position = aligned["Position"].shift(1).fillna(0)

gross_returns = applied_position * aligned["Return"]

position_changes = applied_position.diff().abs().fillna(applied_position.abs())
costs = position_changes * transaction_cost
strategy_returns = gross_returns - costs
equity_curve = (1 + strategy_returns).cumprod()
```

Listing 5.5: Position alignment and cost accounting

5.11 Recommendation Layer and Confidence Gate

The predicted class is mapped to an action, but not unconditionally.

```
MIN_CONFIDENCE = 0.45

predicted = int(np.argmax(proba))
confidence = float(proba[predicted])

# Below the threshold the direction is still reported, but the action
# becomes HOLD. A three class model over noisy daily returns routinely
# puts its leading class at 36 percent, which is technically a winner
# and far too weak to justify taking a position.
acted = predicted if confidence >= min_confidence else 1   # 1 = HOLD
```

Listing 5.6: The confidence gate

The gate addresses a specific failure mode. With three classes, a model carrying no information still assigns approximately one third of probability mass to its leading class. A leading class probability of thirty six percent is therefore barely distinguishable from indifference, yet an unconditional argmax would convert it into a directional instruction. Requiring forty five percent before acting causes the system to decline frequently, which is the intended behaviour. The predicted direction continues to be displayed alongside the downgraded action so that the user can observe what the model inclined towards and how weakly.

CHAPTER 6

IMPLEMENTATION

6.1 Pipeline Orchestration

The pipeline is driven by a command line entry point accepting an instrument key, a model specification, and flags controlling download behaviour and whether training should be performed at all.

```
python -m src.run_pipeline --ticker SBIN --model all
python -m src.run_pipeline --ticker NIFTY50 --model rf,xgb --skip-download
python -m src.run_pipeline --ticker ITC --prepare-only --force
```

Listing 6.1: Pipeline invocation

A subtlety arises when only a subset of models is trained. The metrics file accumulates the performance of every model family, and a naive implementation would overwrite it with only the families just fitted. A twenty second run of the two tree based models would then erase an LSTM result that had taken four minutes to produce. The implementation preserves rows belonging to families not present in the current run.

```
metrics_path = config.RESULTS_DIR / f"{instrument.slug}_metrics.csv"
if metrics_path.exists():
    previous = pd.read_csv(metrics_path)
    fresh = set(metrics["model"])
    kept = previous[~previous["model"].isin(fresh)]
    metrics = pd.concat([metrics, kept], ignore_index=True)
metrics.to_csv(metrics_path, index=False)
```

Listing 6.2: Preserving metric rows for untouched model families

6.2 Dashboard Architecture

The dashboard is implemented using Streamlit, which re-executes the entire script on every user interaction. This execution model simplifies state management but makes caching essential, since an uncached implementation would reload model artefacts and parse data files on every click.



Figure 6.1: Main dashboard showing the prediction for the next session

The recommendation panel presents the action in large type, accompanied by the predicted direction, the confidence value and the reason for any downgrade. The probability distribution over the three classes is rendered as a set of proportional bars with a marker indicating the position of the confidence threshold, so that a user can see at a glance whether the leading class cleared it. A summary of the most recently settled prediction is displayed alongside, reporting what the model stated, what the market subsequently did, and whether the call was correct. This panel always shows the most recent settled observation and is not selected for favourability.

Table 6.1: Dashboard views and their data dependencies

View	Content	Reads from
TODAY	Current recommendation, probabilities, price and RSI charts	Live model artefact, feature matrix
TRACK RECORD	Settled predictions against outcomes, rolling accuracy	Sealed test predictions, labelled data
MODELS	Comparative accuracy, metrics table, equity curves, confusion matrix	Metrics file, prediction files, confusion files
HOW IT WORKS	Pipeline diagram and methodology in plain language	Static content
SCREENER	RSI screen, Bollinger breakouts, band chart	Feature matrices of all instruments

6.3 Instrument Universe Management

The constituent list of the NIFTY 50 index is not fixed. Membership is reviewed periodically and companies are added and removed. Hard coding the list in source would therefore guarantee that it becomes incorrect. The list is instead maintained as a data file, refreshed by a dedicated module.

```
def _load_universe() -> list[Instrument]:
```

```
    """Index constituents from disk. A missing or malformed file is not
    fatal: the application falls back to the curated instruments rather
    than refusing to start."""
    try:
        rows = json.loads(NIFTY50_FILE.read_text(encoding="utf-8"))
    except (OSError, ValueError):
        return []

    out: list[Instrument] = []
    for row in rows:
        symbol = str(row.get("symbol", "")).strip().upper()
        name = str(row.get("name", "")).strip()
        if not symbol or not name:
            continue
        out.append(Instrument(_key_for(symbol), f"{symbol}.NS", name,
                              "stock", _STOCK_THRESHOLD))
    return out
```

Listing 6.3: Loading index constituents from a data file

A separate bulk preparation module acquires, cleans, derives and labels data for every instrument without training any models. It is resumable, and a failure on one symbol does not abort the remainder. It additionally records the span of history available for each instrument, because several constituents listed after 2015 and therefore cannot support the early validation folds. In the preparation run reported here, fifty one of fifty one instruments were prepared successfully, of which forty eight possessed full history and three were flagged as having partial history.

6.4 On Demand Model Training

Preparing data for all instruments in advance while training only on demand is a deliberate division. Measured training times, reported in Chapter 7, show that the recurrent model accounts for approximately ninety three percent of the total cost of fitting all three families. Training every family for every instrument in advance would consume several hours and produce model artefacts occupying several hundred megabytes, the majority of which would never be consulted.

The interface therefore offers two training modes when an instrument has no models. A quick mode fits the two tree based families in approximately fifteen to eighteen seconds. A full mode additionally fits the recurrent model and requires approximately four minutes. Figure 6.2 shows the state presented for an instrument that has data prepared but no models fitted.



Figure 6.2: Untrained instrument state offering quick and full training

The panel reports the span of available history before any training is attempted, so that a user is not left waiting on a training run that cannot succeed. Where an instrument listed after the sealed test window opens, the interface states that the history is insufficient for an honest evaluation.

6.5 The Loading Interface

Training is launched as a separate operating system process. The output of that process is redirected to a log file rather than to a pipe. This is not an arbitrary choice: the deep learning framework emits a substantial volume of diagnostic output, and a pipe of fixed buffer size will fill, at which point the child process blocks on writing while the parent process blocks awaiting the child, producing a deadlock. A log file has no such limit.

```
env = {**os.environ,
       "PYTHONUNBUFFERED": "1",
       "PYTHONIOENCODING": "utf-8",
       "TF_CPP_MIN_LOG_LEVEL": "3"}
```

Listing 6.4: Environment configured for the launched training process

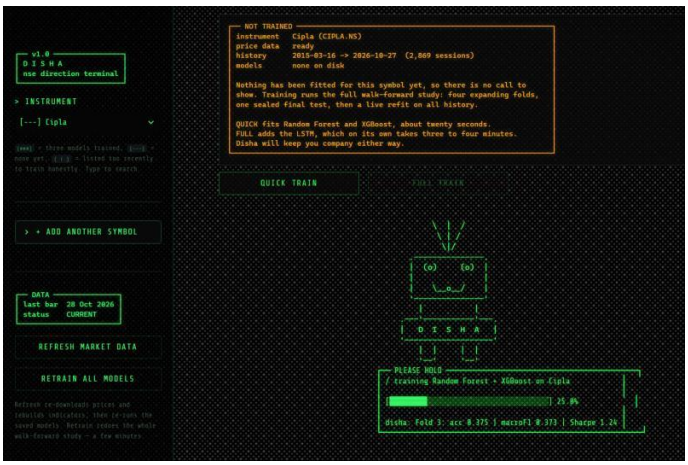


Figure 6.3: Disha loading interface during a training run

The progress indication is derived from genuine stage completion markers rather than from elapsed time, so the reported percentage corresponds to work actually completed. A non zero exit status from the launched process causes the final portion of the log to be surfaced in the interface rather than the failure passing silently.

6.6 Multi Resolution Charting

The price panel supports eleven timeframe presets spanning one minute bars through to five years of weekly bars. Daily and slower resolutions are resampled from data already held locally. Intraday resolutions are not stored and are retrieved on demand from the data provider, which imposes hard limits on intraday history: one minute bars are available for approximately seven days, five minute bars for sixty days and hourly bars for two years. Each preset requests a span within its own limit and fetches a longer history than it displays, so that moving averages are already warm at the left edge of the chart.



Figure 6.4: Price chart with moving averages and relative strength index

Because the models are fitted exclusively on daily bars, intraday resolutions carry no associated prediction. The interface states this explicitly on those views rather than allowing the juxtaposition of a one minute chart and a daily recommendation to imply a relationship that does not exist.

6.7 Technical Screener

Two screens are provided. The first filters instruments by their latest relative strength index value against a user specified level, in either direction. The second identifies instruments whose closing price has breached a Bollinger band within a user specified

number of recent sessions, with the band period and standard deviation multiplier both configurable.

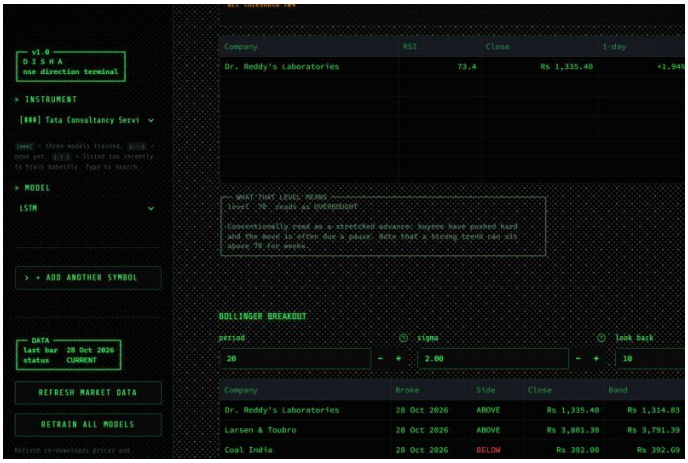


Figure 6.5: Technical screener showing the RSI screen and breakout table

```
def bollinger(close: pd.Series, period: int, n_std: float) -> pd.DataFrame:
    mid = close.rolling(period).mean()
    sd = close.rolling(period).std(ddof=0)
    return pd.DataFrame({"mid": mid,
                         "upper": mid + n_std * sd,
                         "lower": mid - n_std * sd})
```

Listing 6.5: Bollinger band computation with configurable parameters

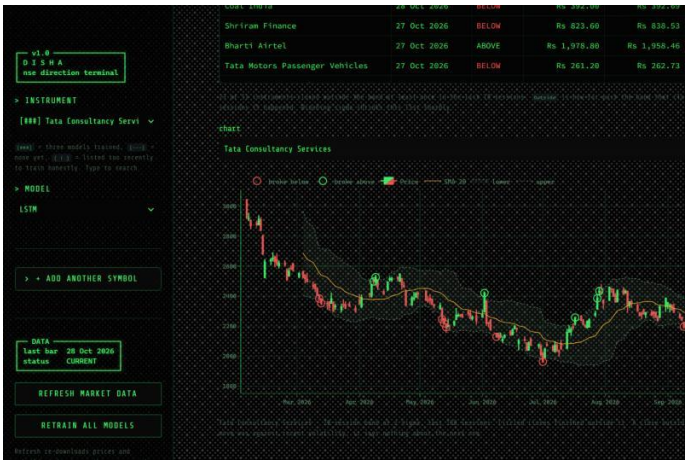


Figure 6.6: Bollinger band breakout chart with breaching closes circled

The screener also presents the conventional interpretation of each relative strength index zone. This material is labelled within the interface as textbook interpretation rather than as a finding of this project, and each zone description includes its own counter case, noting for instance that a strongly trending instrument may remain above the conventional overbought level for an extended period. The distinction is maintained because the measured results of this project do not support treating these readings as predictive.

6.8 Caching Strategy

Derived artefacts are cached and keyed on the modification timestamps of the files from which they derive.

```
@st.cache_resource(show_spinner=False)
def _prediction(ticker: str, model_key: str,
                data_mtime: float, model_mtime: float) -> dict:
    return predict_latest(config.get_instrument(ticker), model_key)


def cached_prediction(inst, model_key: str) -> dict:
    """Keyed on both the data and the model file, so that retraining
    invalidates the cache automatically."""
    return _prediction(
        inst.key, model_key,
        inst.features_path().stat().st_mtime,
        model_stem(inst, model_key, "live").with_suffix(".joblib").stat().st_mtime,
    )
```

Listing 6.6: Modification time keyed caching

The two timestamp arguments participate in the cache key but are unused within the function body. This is the entire mechanism: retraining rewrites the model artefact, its modification time changes, the key changes, and the stale entry is discarded without any explicit invalidation call. It is therefore not possible for the interface to present a recommendation produced by a model that has since been replaced.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Exploratory Data Analysis

Table 7.1: Class balance of the NIFTY 50 labelled dataset

Class	Proportion	Interpretation
UP	39.7 percent	Next session closed more than 0.25 percent higher
DOWN	33.3 percent	Next session closed more than 0.25 percent lower
NEUTRAL	27.0 percent	Next session closed within the threshold band

The exploratory analysis recorded several further observations that informed subsequent modelling decisions.

1. The year 2020 is an outlier in class composition, with the UP proportion rising substantially and the NEUTRAL proportion falling during the pandemic recovery. Fold 1, which validates on 2020, was therefore expected to produce metrics that differ from the remaining folds for reasons of regime rather than model quality.
2. Several derived features are mathematically redundant. The Bollinger middle band is identical to the twenty session simple moving average by construction, exhibiting a correlation of 1.000, and several moving average pairs at the same window exceed 0.99. This redundancy is tolerable for tree based models, which select among correlated features without penalty, but is a candidate for reduction in the recurrent model input.
3. No single technical indicator separates the three classes to any useful degree. The conditional distributions of the relative strength index, the convergence indicator, band width, average true range, momentum and volatility overlap heavily across DOWN, NEUTRAL and UP. This is consistent with the premise of the research question: if a single indicator separated the classes, no model would be required.
4. The India VIX tracks realised volatility closely, with a correlation of 0.836 against twenty session realised volatility, and the two series rise together sharply during the 2020 decline. This supports its retention as a volatility feature.

5. The USD to INR exchange rate exhibits a weak and unstable relationship with the index, with a full period correlation of negative 0.148 and a rolling correlation moving between approximately negative 0.4 and positive 0.4. It was retained as a feature but was not expected to contribute strongly.

7.2 Walk Forward Fold Results

Table 7.2 reports per fold validation results for the NIFTY 50 index across the three model families.

Table 7.2: Walk forward fold results for the NIFTY 50 index

Model	Fold	Observations	Accuracy	Macro F1	Sharpe
Random Forest	Fold 1 (2020)	250	0.3680	0.3552	-0.27
Random Forest	Fold 2 (2021)	248	0.3790	0.3659	-1.26
Random Forest	Fold 3 (2022)	248	0.3548	0.3451	-1.18
Random Forest	Fold 4 (2023)	245	0.4122	0.3866	0.63
XGBoost	Fold 1 (2020)	250	0.3480	0.3469	0.74
XGBoost	Fold 2 (2021)	248	0.3750	0.3582	-0.94
XGBoost	Fold 3 (2022)	248	0.3508	0.3441	-0.43
XGBoost	Fold 4 (2023)	245	0.4000	0.3467	0.25
LSTM	Fold 1 (2020)	250	0.3560	0.3137	-0.71
LSTM	Fold 2 (2021)	248	0.3911	0.3562	1.28
LSTM	Fold 3 (2022)	248	0.3306	0.2475	-0.38
LSTM	Fold 4 (2023)	245	0.3755	0.3596	0.81

Accuracy varies by roughly six percentage points across folds for every model family, from 0.3306 to 0.4122. This spread is larger than the spread between model families within any single fold, which is the first indication that the choice of evaluation period matters more than the choice of algorithm. Fold 3, validating on the 2022 rate tightening period, is the weakest fold for all three families, and Fold 4 is the strongest for all three. A model selected on the basis of one fold alone could therefore be reported as substantially better or worse than it is.

7.3 Sealed Test Set Results

Table 7.3 reports the results of the single sealed evaluation for the NIFTY 50 index.

Table 7.3: Sealed test set results, NIFTY 50 index (689 observations)

Model	Accuracy	Macro F1	Sharpe	CAGR	Max drawdown	Trades
Random Forest	0.3875	0.3832	0.44	4.76 percent	-21.4 percent	233

Model	Accuracy	Macro F1	Sharpe	CAGR	Max drawdown	Trades
XGBoost	0.3585	0.3377	-1.56	-17.64 percent	-42.0 percent	224
LSTM	0.3599	0.3276	0.13	0.82 percent	-17.0 percent	38
Always UP (baseline)	0.3687	0.1796	0.22	2.06 percent	-15.8 percent	1
Buy and Hold	Not applicable	Not applicable	0.22	2.06 percent	-15.8 percent	1

On the index, only Random Forest exceeds the majority class baseline on accuracy, by 1.9 percentage points. The standard error of the baseline proportion on 689 observations is approximately 1.84 percentage points, so this difference amounts to roughly one standard error and cannot be distinguished from sampling variation. XGBoost and the LSTM both fall below the baseline, though again within one standard error.

The macro averaged F1 column tells a different story. All three models score between 0.33 and 0.38 against the baseline's 0.1796. This is not an artefact: a constant classifier necessarily scores zero F1 on the two classes it never predicts, so its macro average is structurally low. The appropriate comparison for macro F1 is uniform random guessing, which on a three class problem with roughly balanced classes would yield approximately 0.33. Random Forest at 0.3832 exceeds that, which indicates that the model is recovering some genuine class structure even where accuracy does not separate it from the baseline.

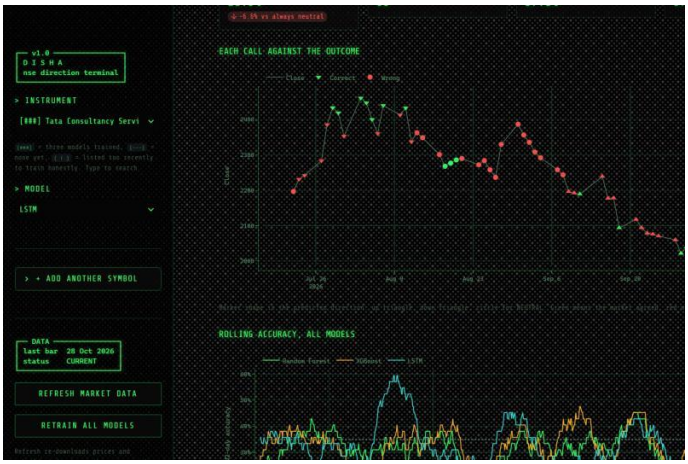


Figure 7.1: Track record of settled calls against realised outcomes



Figure 7.2: Model comparison on the sealed test set

Table 7.4 extends the comparison to all eight instruments for which the full three model suite was trained.

Table 7.4: Sealed test set accuracy across all eight instruments

Instrument	Random Forest	XGBoost	LSTM	Baseline	Baseline rule
NIFTY 50	0.3875	0.3585	0.3599	0.3687	Always UP
TCS	0.3333	0.3333	0.3188	0.3493	Always NEUTRAL
Reliance	0.3638	0.3536	0.3406	0.3130	Always UP
Infosys	0.3130	0.3420	0.3507	0.3333	Always UP
HDFC Bank	0.3652	0.3696	0.3551	0.3797	Always NEUTRAL
ICICI Bank	0.4087	0.4246	0.4203	0.3130	Always UP
State Bank of India	0.3710	0.3609	0.3478	0.3464	Always UP
ITC	0.3870	0.4014	0.3957	0.2652	Always UP

The secondary research question concerning index against individual stock can be addressed from this table. The index is not systematically easier to predict than its constituents. The two strongest results in the study, ICICI Bank and ITC, are individual stocks, while two constituents, TCS and HDFC Bank, fall below their baselines for every model family. The variation across instruments is substantially larger than the variation across model families.

7.4 Statistical Significance Against the Baseline

Reporting raw accuracy differences without reference to sampling variation would be misleading at these sample sizes. For each instrument the standard error of the baseline proportion was computed as the square root of p multiplied by one minus p divided by n, where p is the baseline accuracy and n the number of test observations. Standard

errors fall between 1.68 and 1.85 percentage points across the eight instruments. Table 7.5 reports the difference between each model and its baseline expressed in standard errors.

Table 7.5: Statistical comparison against the majority class baseline

Instrument	Model	Gap (percentage points)	Standard errors
TCS	LSTM	-3.04	-1.68
HDFC Bank	LSTM	-2.46	-1.33
Infosys	Random Forest	-2.03	-1.13
TCS	Random Forest	-1.59	-0.88
TCS	XGBoost	-1.59	-0.88
HDFC Bank	Random Forest	-1.45	-0.78
HDFC Bank	XGBoost	-1.01	-0.55
NIFTY 50	XGBoost	-1.02	-0.55
NIFTY 50	LSTM	-0.87	-0.47
State Bank of India	LSTM	0.14	0.08
Infosys	XGBoost	0.87	0.48
State Bank of India	XGBoost	1.45	0.80
Infosys	LSTM	1.74	0.97
NIFTY 50	Random Forest	1.89	1.03
State Bank of India	Random Forest	2.46	1.36
Reliance	LSTM	2.75	1.56
Reliance	XGBoost	4.06	2.30
Reliance	Random Forest	5.07	2.87
ICICI Bank	Random Forest	9.57	5.42
ICICI Bank	LSTM	10.72	6.07
ICICI Bank	XGBoost	11.16	6.32
ITC	Random Forest	12.17	7.24
ITC	LSTM	13.04	7.76
ITC	XGBoost	13.62	8.11

Summarising the twenty four comparisons: sixteen fall within two standard errors of the baseline and are therefore statistically indistinguishable from it, eight exceed it by more than two standard errors, and none falls more than two standard errors below it. The median comparison stands at 0.885 standard errors, which is positive but not significant.

The eight significant results are concentrated in three instruments, namely ITC, ICICI Bank and Reliance. This concentration requires careful interpretation rather than celebration. In each of those three cases the applicable baseline is Always UP with a low base rate: 26.5 percent for ITC and 31.3 percent for both ICICI Bank and Reliance. Where no single class dominates, the majority class baseline is itself weak and

correspondingly easier to exceed. The models in these cases are not demonstrating unusual skill so much as benefiting from a less demanding comparison.

The appropriate conclusion is therefore conditional. Statistically significant improvement over the majority class baseline was obtained on three of eight instruments, and those three share the characteristic of a diffuse class distribution. On the remaining five, including the index itself, the models cannot be distinguished from a rule requiring no model at all.

7.5 Confusion Matrix Analysis

Accuracy alone conceals the structure of a classifier's errors. Table 7.6 reports the confusion matrix for Random Forest on the NIFTY 50 sealed test set.

Table 7.6: Confusion matrix, Random Forest on the NIFTY 50 sealed test set

Actual \ Predicted	DOWN	NEUTRAL	UP	Total	Recall
DOWN	99	79	55	233	42.5 percent
NEUTRAL	59	106	37	202	52.5 percent
UP	97	95	62	254	24.4 percent

This matrix is more informative than the accuracy figure that summarises it. The model identifies NEUTRAL sessions most reliably, at 52.5 percent recall, and DOWN sessions reasonably at 42.5 percent. Its recall on UP sessions is 24.4 percent, meaning that on three quarters of the sessions that actually rose, the model predicted something else. Of the 254 genuine UP sessions, 97 were classified as DOWN.

7.6 Trading Performance After Costs

Classification accuracy and trading profitability are distinct questions. Table 7.7 reports the Sharpe ratio of each model strategy against buy and hold for the same instrument and period.

Table 7.7: Risk adjusted performance against buy and hold (annualised Sharpe)

Instrument	Random Forest	XGBoost	LSTM	Buy and Hold	Models ahead
NIFTY 50	0.44	-1.56	0.13	0.22	1 of 3
TCS	-0.51	-0.39	-0.55	-0.46	1 of 3
Reliance	-0.11	-1.13	0.30	0.02	1 of 3
Infosys	-1.29	-0.64	-0.02	-0.21	1 of 3
HDFC Bank	-1.03	-1.15	-0.20	-0.09	0 of 3
ICICI Bank	-0.43	0.28	0.14	0.76	0 of 3

Instrument	Random Forest	XGBoost	LSTM	Buy and Hold	Models ahead
State Bank of India	-0.12	-0.64	0.16	0.80	0 of 3
ITC	-0.31	0.41	0.44	-0.69	3 of 3

Only seven of twenty four model and instrument combinations exceed buy and hold on risk adjusted return after transaction costs. Three of those seven occur on ITC, where buy and hold itself produced a negative Sharpe of -0.69 over the test period, so exceeding it is not a demanding achievement.

This observation is central. A model may be more accurate and still less profitable, because accuracy weights all sessions equally whereas returns weight them by the magnitude of price movement, and because every change of position incurs a cost that accuracy does not register.

7.7 Effect of Model Complexity

The primary research question asks whether increasing model complexity improves performance. Table 7.8 aggregates across all eight instruments.

Table 7.8: Mean performance by model family across eight instruments

Model family	Mean accuracy	Mean gap to baseline	Mean macro F1	Instruments above baseline
Random Forest	0.3662	+3.26 points	0.3249	5 of 8
XGBoost	0.3680	+3.44 points	0.3119	5 of 8
LSTM	0.3611	+2.75 points	0.3170	5 of 8

The three families are separated by 0.69 percentage points of mean accuracy, from 0.3611 to 0.3680. Standard errors on individual instruments are between 1.68 and 1.85 percentage points, so this spread is well inside the noise floor. Each family exceeds its baseline on exactly five of eight instruments. On this evidence the three are indistinguishable.

The ordering that does appear is not the ordering the complexity hypothesis predicts. XGBoost, the middle member of the ladder, records the highest mean accuracy. The LSTM, the most structurally sophisticated model and the only one capable of representing temporal dependence directly, records the lowest. If modelling sequence explicitly conferred an advantage at a one day horizon on this feature set, the LSTM should lead. It does not.

The answer to the primary research question, on this evidence, is that increasing model complexity from Random Forest through XGBoost to LSTM does not improve predictive accuracy or trading performance on the Indian stock market under a temporally correct validation scheme.

7.8 Computational Cost

Training cost was measured directly on the development machine using the TCS dataset of 2,172 training observations and twenty eight features. Each walk forward study requires six fits per model family, comprising four validation folds, one sealed test fit and one refit on all history.

Table 7.9: Measured training time per model family

Model family	Time per fit	Six fits	Share of total
Random Forest	2.4 seconds	14.2 seconds	6.1 percent
XGBoost	0.4 seconds	2.6 seconds	1.1 percent
LSTM	36.2 seconds	217.1 seconds	92.8 percent
All three families	Not applicable	233.9 seconds	100 percent

The recurrent model consumes 92.8 percent of total training time while producing the lowest mean accuracy of the three families. The cost ratio against XGBoost is approximately eighty three to one. This is a practical result as well as an academic one: it directly motivated the division of the application into a quick training mode fitting the two tree based families in approximately fifteen to eighteen seconds, and a full mode requiring approximately four minutes, allowing a user to obtain a usable result on a new instrument in under twenty seconds.

7.9 Consolidated Observations

The following observations summarise the findings of the study.

1. Fold to fold variation exceeds model to model variation. Accuracy for a single model family varies by approximately six percentage points across validation years, while the three families differ by less than one percentage point on average. The evaluation period matters more than the algorithm.
2. Sixteen of twenty four model and instrument combinations are statistically indistinguishable from a majority class baseline. Eight exceed it significantly and none falls significantly below it.

3. Only seven of twenty four combinations exceed buy and hold on risk adjusted return after transaction costs, and three of those occur where buy and hold itself lost money.
4. Accuracy and profitability are weakly related. The least accurate family achieves the best Sharpe ratio on half the instruments, because it trades far less frequently and therefore incurs lower costs.
5. Increasing model complexity does not improve results. The ordering observed is XGBoost, then Random Forest, then LSTM, which is not the ordering the complexity hypothesis predicts.
6. Individual stocks are not uniformly harder to predict than the index. The strongest and weakest results in the study both occur on individual stocks.

CHAPTER 8

CONCLUSION

8.1 Summary of Work

8.2 Answer to the Research Question

The primary research question asked whether increasing model complexity from Random Forest through XGBoost to Long Short Term Memory networks improves predictive accuracy and trading performance on the Indian stock market.

The answer obtained is that it does not. Mean accuracy across eight instruments separates the three families by 0.69 percentage points against standard errors of between 1.68 and 1.85 percentage points, placing all three within the noise floor of one another. Each family exceeds its baseline on exactly five of eight instruments. The ordering that emerges places XGBoost first and the LSTM last, which inverts the ordering the complexity hypothesis would predict, and the LSTM consumes approximately ninety three percent of total training time to achieve that last position.

A more precise statement of the overall result is warranted, since it is neither uniformly negative nor straightforwardly positive. Of twenty four model and instrument combinations, sixteen are statistically indistinguishable from a majority class baseline, eight exceed it by more than two standard errors, and none falls significantly below it. The eight significant results concentrate on three instruments whose class distributions are diffuse and whose baselines are consequently weak, which means they reflect an easier comparison rather than a demonstrated general capability. Macro averaged F1 at approximately 1.91 times the constant baseline confirms that the models learn real class structure. That structure, however, does not convert into trading profitability: only seven of twenty four combinations exceed buy and hold on risk adjusted return once a transaction cost of 0.05 percent per position change is charged.

The contribution of this work is therefore methodological rather than predictive. It demonstrates that when temporal validation, a sealed test set, an honest majority class baseline and transaction costs are applied simultaneously, the large apparent predictability of next day direction reported in much of the applied literature largely disappears, and what remains is modest, instrument specific and not reliably tradeable.

8.3 Limitations

The following limitations bound the conclusions that may be drawn from this work.

Horizon. The study examines a single trading day horizon exclusively. Predictability absent at one day may exist at five, ten or twenty days, and the labelling function would require only a parameterised shift to test this.

Feature scope. Only price derived technical features and two macroeconomic series were used. News sentiment, order book data, corporate announcements, analyst revisions and sectoral rotation were not incorporated. The conclusion is specific to the information set employed.

Execution assumptions. The backtest assumes execution at the closing price on the signal day, unlimited short selling, and a flat transaction cost of 0.05 percent. Slippage, market impact, borrowing cost and the separate components of Indian transaction charges are not modelled.

Instrument coverage. The full three model comparison was conducted on eight instruments. The remaining forty three constituents have prepared data and can be trained on demand, but were not included in the comparative study.

Survivorship. The constituent list is the current membership of the index. Companies removed from the index during the study period are absent, which introduces a survivorship consideration into any cross sectional interpretation.

Partial histories. Three constituents listed after 2015 and therefore cannot populate the earlier validation folds. The system reports this condition but does not otherwise compensate for it.

Single evaluation window. The sealed test covers January 2024 onward. A different test window could produce different results, and the statistical comparisons reported here use a normal approximation to the binomial rather than an exact test or a bootstrap interval.

8.4 Future Scope

Several directions follow naturally from the limitations above and from the findings of the study.

1. Horizon sweep. Parameterise the labelling shift and evaluate horizons of one, five, ten and twenty sessions under the identical validation scheme. If predictability emerges at longer horizons this is a positive finding, and if it does not the negative result becomes considerably stronger.
2. Replication of the shuffled split. Implement a randomised train and test split on the same data and the same models, report the accuracy obtained, and contrast it directly with the walk forward result. This would quantify the inflation attributable to the validation design alone and would convert a methodological argument into a measured one.
3. Bootstrap confidence intervals. Replace the normal approximation used in Section 7.4 with bootstrap intervals on both accuracy and Sharpe, which would place the conclusions on a firmer inferential footing and permit significance testing of the differences between model families.
4. Feature ablation. Evaluate price only features against price plus macroeconomic features against the full set including market context, in order to identify which categories of information carry signal.
5. Regime conditional analysis. Partition the test window by volatility regime using the India VIX and evaluate whether any model exceeds its baseline within a particular regime even where it does not do so overall.
6. Hybrid architecture. Investigate the combination of recurrent and boosted models explored in the reviewed literature, in which a recurrent encoder supplies learned representations to a gradient boosted classifier.

BIBLIOGRAPHY

[1] Md. Arif Istiake Sunny, Mirza Mohd Shahriar Maswood and Abdullah G. Alharbi, *Deep Learning-Based Stock Price Prediction Using LSTM and Bi-Directional LSTM Model*.

[2] Hassan Oukhouya, Hamza Kadiri, Khalid El Himdi and Raby Guerbaz, *Forecasting International Stock Market Trends: XGBoost, LSTM, LSTM-XGBoost, and Backtesting XGBoost Models*.

[3] Ishita Khandelwal, Suchita Borkar and Prakash Ukhalkar, *Comparative Analysis of LSTM, Random Forest, and XGBoost Models for Stock Price Prediction in the Indian Equity Market*.

[4] Hasan Ahmed Salman, Ali Kalakech and Amani Steiti, *Random Forest Algorithm Overview*.

[5] Ramraj S, Nishant Uzir, Sunil R and Shatadeep Banerjee, *Experimenting XGBoost Algorithm for Prediction and Classification of Different Datasets*.

[6] Ugur Sahin and A. Murat Ozbayoglu, *Trend-normalized RSI Indicator for Stock Trading Systems with Evolutionary Computation*.

[7] https://finance.yahoo.com Accessed 28th October 2026.

[8] https://en.wikipedia.org/wiki/NIFTY_50 Accessed 28th October 2026.

[9] https://scikit-learn.org/stable/modules/ensemble.html Accessed 28th October 2026.

[10] https://xgboost.readthedocs.io Accessed 28th October 2026.

[11] https://keras.io/api/layers/recurrent_layers/lstm Accessed 28th October 2026.

[12] https://docs.streamlit.io Accessed 28th October 2026.